

GRouNdGAN Generator Scaling in the Original Implementation

Analytical analysis of `upstream/GRouNdGAN`

September 23, 2026

1 Purpose

This report derives the causal-generator parameter-count scaling for the original upstream implementation in `/mnt/shared/scratch/mmicik/private/Geneformer/simulation/upstream/GRouNdGAN`. It is intentionally separate from the corresponding analysis for the `dedup` implementation.

Assume every target gene has exactly k regulators, so the imposed-edge count is

$$E = kG, \tag{1}$$

where G is the number of target genes.

2 Definitions

Table 1: Symbols used in this report.

Symbol	Meaning
F	Number of transcription factors
G	Number of target genes
$N = F + G$	Total genes, assuming disjoint TF and target sets
k	Regulators selected per target gene
$E = kG$	Number of imposed edges
n	Noise variables per target gene
w	Width scale per gene
d	Depth per gene
$q = k + n$	Regulators plus noise variables per target
$h = wq$	Hidden width per target-gene block
$H = Gh$	Total hidden width
$X = F + nG$	Input width of the first masked layer

The upstream implementation uses dense tensors whose weights are masked at forward and backward time. Therefore, trainable-parameter counts and checkpoint/state-dict tensor counts are different.

3 Original Masked Generator

In the upstream code, `MaskedLinear` creates a dense weight tensor with shape `(output_features, input_features)` and separately stores the mask. The mask zeros weights and gradients but does

not make the weight tensor sparse.

The three masked layers have shapes:

$$\text{input layer: } X \times H, \quad (2)$$

$$\text{hidden layer: } H \times H, \quad (3)$$

$$\text{output layer: } H \times G. \quad (4)$$

There is one input layer, d hidden layers, and one output layer.

3.1 Trainable Parameters

Counting dense weights, biases, and BatchNorm affine parameters, the causal-graph-specific generator has

$$P_{\text{graph,train}} = HX + dH^2 + HG + (3d + 3)H + G. \quad (5)$$

The terms are respectively dense weights, masked-layer biases, and BatchNorm affine parameters.

3.2 Checkpoint/State-Dict Tensor Count

The upstream mask is an `nn.Parameter` with `requires_grad=False`, so it is present in the state dictionary. BatchNorm also stores running mean, running variance, and a batch counter. Counting those tensors gives

$$P_{\text{graph,state}} = 2HX + 2dH^2 + 2HG + 5(d + 1)H + (d + 1) + G. \quad (6)$$

This is the appropriate expression when comparing against a checkpoint-inspection result that counts every tensor in the state dictionary rather than only trainable tensors.

4 Causal Controller

Let the causal-controller latent dimension be z , its hidden widths be $a_1, \dots, a_m$, and $a_0 = z$. Its output dimension is $N = F + G$.

The trainable controller count is

$$P_{\text{CC,train}} = \sum_{j=1}^m [(a_{j-1} + 1)a_j + 2a_j] + (a_m + 1)(F + G). \quad (7)$$

For state-dict counting, each hidden BatchNorm additionally contributes its running statistics and counter:

$$P_{\text{CC,state}} = \sum_{j=1}^m [(a_{j-1} + 1)a_j + 4a_j + 1] + (a_m + 1)(F + G). \quad (8)$$

If the generator state dictionary includes the pretrained controller, the full original-generator count is

$$P_{\text{gen,state}} = P_{\text{CC,state}} + P_{\text{graph,state}}. \quad (9)$$

For trainable parameters only, replace both state terms with their trainable counterparts.

5 Partial Scaling Functions

The following expressions hold with all variables other than the named variable fixed.

5.1 TF Count F

Because $X = F + nG$ and the controller output is $F + G$:

$$\boxed{P_{\text{graph,state}}(F) = 2HF + C_F,} \quad (10)$$

where C_F is independent of F . Thus, the upstream state-dict count is linear in F .

5.2 Target-Gene Count G

With F, k, d, w, n fixed, $H = Gw(k + n)$ and $X = F + nG$. Therefore:

$$\boxed{P_{\text{graph,state}}(G) = 2dw^2(k + n)^2G^2 + 2w(k + n)(F + nG)G + 2w(k + n)G^2 + 5(d + 1)w(k + n)G + (d + 1) + G.} \quad (11)$$

Unlike the sparse implementation, the original dense-mask implementation is quadratic in G .

5.3 Top- k

With F, G, d, w, n fixed and $q = k + n$:

$$\boxed{P_{\text{graph,state}}(k) = 2G^2dw^2(k + n)^2 + 2Gw(k + n)(F + nG + G) + 5(d + 1)Gw(k + n) + (d + 1) + G.} \quad (12)$$

Thus, the original implementation is quadratic in k .

5.4 Depth d

With F, G, k, w, n fixed:

$$\boxed{P_{\text{graph,state}}(d) = 2G^2w^2(k + n)^2d + 5Gw(k + n)d + C_d,} \quad (13)$$

where C_d is independent of d . Depth is linear.

5.5 Width w

With F, G, k, d, n fixed:

$$\boxed{P_{\text{graph,state}}(w) = 2dG^2(k + n)^2w^2 + [2G(k + n)(F + nG + G) + 5(d + 1)G(k + n)]w + C_w.} \quad (14)$$

Width is quadratic because the dense hidden matrices scale as H^2 .

5.6 Noise Per Gene n

With F, G, k, d, w fixed, both $H = wG(k + n)$ and $X = F + nG$ vary with n :

$$\boxed{P_{\text{graph,state}}(n) = 2dG^2w^2(k + n)^2 + 2Gw(k + n)(F + nG + G) + 5(d + 1)Gw(k + n) + (d + 1) + G.} \quad (15)$$

Noise count is quadratic.

5.7 Imposed Edges E

Substituting $k = E/G$ gives $H = w(E + nG)$ and

$$\begin{aligned}
 P_{\text{graph,state}}(E) = & 2dw^2(E + nG)^2 \\
 & + 2w(E + nG)(F + nG + G) \\
 & + 5(d + 1)w(E + nG) + (d + 1) + G.
 \end{aligned}
 \tag{16}$$

For fixed F, G, d, w, n :

$$P_{\text{graph,state}}(E) = AE^2 + BE + C. \tag{17}$$

The quadratic coefficient is $2dw^2$ for the state-dict count. For trainable parameters it is dw^2 because the fixed mask tensors are excluded.

6 Significant Differences from dedup

Table 2: Implementation differences that change the scaling equations.

Feature	Original upstream implementation	dedup implementation
Masked layer	Dense MaskedLinear weight matrices; masks zero weights and gradients.	Connectivity-based SparseLinear ; only selected connections receive weight parameters.
TF dependence	The input matrix has shape $(F + nG) \times H$, so F directly affects parameter count.	TF count mostly affects connectivity indices; it does not multiply the stored sparse weights for fixed k .
State-dict masks	Masks are <code>nn.Parameter(requires_grad=False)</code> and are saved in checkpoints.	Connectivity is stored as index <code>False</code> ; dense mask tensors are not stored as model parameters.
Generator contents	CausalGenerator creates an internal <code>_labeler</code> .	The deduplicated causal-generator module does not create that internal labeler.
Mask construction	Dense $H \times H$ masks are materialized.	Masks are constructed as sparse CSR matrices and converted to connectivity indices.
Implication	Dense hidden matrices make parameter/state-dict scaling larger and more explicitly dependent on F, G , and H^2 .	Sparse parameter scaling follows the number of nonzero connections, with a different coefficient and weaker TF dependence.

The same logical top- k graph can therefore produce substantially different parameter counts in the two implementations. The upstream equations must not be applied to **dedup** checkpoint counts, and vice versa.

7 Implementation References

The upstream derivation is based on:

- `src/networks/masked_causal_generator.py`, lines 155–231: mask construction and layer layout.
- `src/networks/masked_causal_generator.py`, lines 233–245: internal labeler.
- `src/networks/masked_causal_generator.py`, lines 272–293: masked-layer creation.
- `src/layers/masked_linear.py`, lines 85–108: dense weight and mask allocation.
- `src/layers/masked_linear.py`, lines 110–123: mask application and forward pass.